

Week 6 easier

Week 6: Multi-rate DSP. Downsampling (decimation) and upsampling (interpolation). Frequency domain effects of rate changes.

Week 6: Multi-rate DSP

Lecture 11: Downsampling, Decimation, and CNN Strides *Oppenheim & Schafer: Chapter 4 (4.6) | Raschka: CNN Architecture*

- **0:00 – 0:25 | Downsampling in Time and Frequency:**
 - Time domain: $x_d[n] = x[nM]$.
 - Frequency domain: $X_d(e^{j\omega}) = \frac{1}{M} \sum_{k=0}^{M-1} X(e^{j(\omega - 2\pi k)/M})$.
 - Visualizing the stretching and overlapping (aliasing) of the frequency spectrum.
- **0:25 – 0:50 | Decimation (Filtering + Downsampling):** The necessity of an anti-aliasing lowpass filter before downsampling. Calculating the required cutoff frequency $\omega_c = \pi/M$. Why we use linear phase FIR filters here (preventing phase distortion on the preserved band).
- **0:50 – 1:15 | ML Context – Strided Convolutions and Aliasing:** Comparing classical decimation to `PyTorch Conv1d(..., stride=M)`. Highlighting the modern ML problem: standard strided convolutions often violate the Nyquist-Shannon sampling theorem, causing shift-equivariance loss in CNNs (referencing modern fixes like BlurPool/anti-aliased CNNs).

Lecture 12: Upsampling, Interpolation, and Rational Rate Changes *Oppenheim & Schafer: Chapter 4 (4.6)*

- **0:00 – 0:20 | Upsampling (Zero-Insertion):**
 - Time domain: Inserting $L - 1$ zeros between samples.
 - Frequency domain: $X_i(e^{j\omega}) = X(e^{j\omega L})$. Visualizing spectrum compression and the creation of high-frequency images.
- **0:20 – 0:45 | Interpolation:** Applying an anti-imaging lowpass filter to remove the periodic spectral replicas. Scaling the filter gain by L to preserve time-domain amplitude.
- **0:45 – 1:00 | Rational Rate Changes (L/M):** Cascading interpolation and decimation. Why we always upsample first and downsample second, and how to

combine the anti-imaging and anti-aliasing filters into a single lowpass filter with $\omega_c = \min(\pi/L, \pi/M)$.

- **1:00 – 1:15 | ML Context – Transposed Convolutions:** Mapping interpolation to PyTorch ConvTranspose1d. Explaining the DSP origin of "checkerboard artifacts" in generative audio/image models and how proper anti-imaging filter initialization can prevent them.

Prompt: Can you create slides and python code for these two 75 minute classes of a DSP and ML course for a combined senior and first year Electrical and Computer Engineering class? The DSP textbook is Discrete-Time Signal Processing by Oppenheim and Schaffer. The ML textbook is Machine Learning with PyTorch and Scikit-Learn by Raschka. Add some homework problems with solutions and a weekly quiz with solutions as well.

ChatGPT: (Medium)

Done. I created a complete Week 6 package with two lecture decks, runnable Python material, homework, and a weekly quiz with solutions.

Lecture slides

- [Lecture 11 — Downsampling, Decimation, and CNN Strides](#)
- [Lecture 12 — Upsampling, Interpolation, and Rational Rate Changes](#)

The decks include worked examples, frequency-domain visualizations, conceptual questions, PyTorch/CNN connections, pacing for the 75-minute classes, and instructor notes.

Python material

- [Jupyter notebook — Week 6 Multi-rate DSP and ML Demos](#)
- [Plain Python version](#)

The code covers downsampling and aliasing, FIR decimation, `resample_poly`, strided Conv1d, shift sensitivity, zero insertion, interpolation gain, ConvTranspose1d, and uneven-overlap artifacts. I also executed the notebook successfully as a validation check.

Homework and weekly quiz

- [Homework + Weekly Quiz + Complete Solutions](#)

This contains eight homework problems progressing from fundamentals to senior-level DSP/ML synthesis, an eight-question weekly quiz, full solutions, and grading guidance.

Complete package

- [Download the entire Week 6 package](#)
-

Week 6, Lecture 11

Explain in more detail what's going on here and what good answers and/or examples are. For all mathematical equations, use standard single dollar signs ... for inline math and double dollar signs

...

for block/display math. Do not use (...) or [...]. Output the raw text without escaping the dollar signs.

Chat GPT (Medium):

Below is a more detailed instructor-level explanation of Lecture 11, with emphasis on what is happening conceptually, what students often misunderstand, and what strong answers/examples look like. I will follow the structure of the slide deck closely.

Lecture 11: Downsampling, Decimation, and CNN Strides

The unifying idea of the lecture is that reducing the number of samples in a discrete-time signal is fundamentally a sampling operation. Classical DSP calls this downsampling or decimation. In a CNN, the same mathematical act appears when we use a stride greater than one.

The key conceptual question for the entire lecture is:

What must happen before we throw samples away?

The DSP answer is: remove frequency content that the lower sampling rate cannot represent.

The ML connection is that CNNs frequently throw feature-map samples away through strides or pooling without explicitly enforcing that requirement.

1. Downsampling in the time domain

The mathematical definition is

$$x_d[n] = x[nM]$$

where M is a positive integer.

This means we keep every M th sample of $x[n]$.

If the original sequence is

$$x[0], x[1], x[2], x[3], x[4], x[5], \dots$$

then downsampling by $M = 3$ produces

$$x_d[0] = x[0]$$

$$x_d[1] = x[3]$$

$$x_d[2] = x[6]$$

and so forth.

The important point is that **the amplitudes are not scaled**. We are selecting samples, not dividing values by M .

The deck emphasizes that the sample rate changes from F_s to F_s/M , while no new sample values are calculated.

A useful physical interpretation

Suppose

$$F_s = 48 \text{ kHz}$$

and we downsample by

$$M = 3.$$

The output sampling rate becomes

$$F_{s,\text{out}} = \frac{48000}{3} = 16000 \text{ Hz.}$$

Originally, samples were separated by

$$T_s = \frac{1}{48000}.$$

After downsampling, the spacing is

$$T_{s,\text{out}} = \frac{1}{16000} = 3T_s.$$

So we have not simply deleted array entries. We have created a new sampled-time signal whose samples represent measurements farther apart in physical time.

2. The index-mapping example

The slide uses

$$x[n] = \{2, 1, -1, 3, 4, 0, 5, 2, -2, 1, \dots\}$$

and

$$M = 3.$$

The indices retained are

$$n = 0, 3, 6, 9, \dots$$

so

$$x_d[n] = \{2, 3, 5, 1, \dots\}.$$

Good question to ask students

Which entries survive if $M = 3$?

A good answer is:

We retain samples whose original indices are multiples of 3: 0, 3, 6, 9,

A weaker but still acceptable answer would be:

Keep every third sample starting at $x[0]$.

An incorrect answer would be:

Divide every sample by 3.

That error is worth discussing because the frequency-domain expression does contain a factor $1/M$, which sometimes leads students to incorrectly assume the time-domain amplitudes are scaled.

3. What happens in frequency?

The downsampling relationship is

$$X_d(e^{j\omega}) = \frac{1}{M} \sum_{k=0}^{M-1} X(e^{j(\omega - 2\pi k)/M}).$$

The slide summarizes this process as:

stretch, shift, add.

This is probably the most important frequency-domain mental model in the lecture.

Step 1: Stretch

Consider

$$X(e^{j\omega/M}).$$

If the original signal had a feature centered around normalized frequency ω_0 , it appears at approximately

$$M\omega_0$$

on the new normalized-frequency axis.

So downsampling causes the spectrum to appear wider.

This initially feels backwards to students because the time-domain sequence becomes "shorter."

A useful reminder is:

Compression or loss of samples in time generally corresponds to expansion in frequency.

4. Why are there several spectral copies?

The DTFT of a discrete-time signal is periodic:

$$X(e^{j(\omega+2\pi)}) = X(e^{j\omega}).$$

When we replace ω by something like

$$\frac{\omega - 2\pi k}{M},$$

multiple periodically shifted versions of the original spectrum map into the interval

$$-\pi \leq \omega \leq \pi.$$

Those copies are then added.

That last part is critical.

They are not merely drawn on top of each other for visualization. Mathematically, they are **summed**.

If two pieces overlap, the resulting spectrum contains their sum, and the original pieces can no longer be uniquely separated.

That is aliasing.

5. A simple aliasing example

Suppose the original signal contains

$$x[n] = \cos(0.8\pi n).$$

Now downsample by $M = 2$:

$$x_d[n] = x[2n].$$

Therefore

$$x_d[n] = \cos(0.8\pi(2n)) = \cos(1.6\pi n).$$

Discrete-time frequencies are periodic modulo 2π , so

$$1.6\pi \equiv -0.4\pi.$$

Because cosine is even,

$$\cos(-0.4\pi n) = \cos(0.4\pi n).$$

Therefore

$$x_d[n] = \cos(0.4\pi n).$$

The original frequency 0.8π has appeared at 0.4π after downsampling.

That is a concrete example of frequency folding.

Good student answer

The original sinusoid was above the new Nyquist limit. After downsampling, it becomes indistinguishable from a lower-frequency sinusoid.

That is stronger than merely saying:

It aliases.

The stronger answer explains what aliasing actually means.

6. What is the alias-free condition?

Before downsampling by M , the input should be confined approximately to

$$|\omega| < \frac{\pi}{M}.$$

The slide gives this as the alias-free requirement.

Why?

After downsampling, the new sample rate is

$$\frac{F_s}{M}$$

Therefore the new Nyquist frequency is

$$\frac{F_s}{2M}.$$

In normalized input frequency, this corresponds to

$$\omega = \frac{\pi}{M}.$$

Any energy above that limit will fold into frequencies representable at the new sample rate.

7. Useful numerical example

Suppose

$$F_s = 48 \text{ kHz}$$

and

$$M = 3.$$

Then

$$F_{s,\text{out}} = 16 \text{ kHz}.$$

The output Nyquist frequency is therefore

$$8 \text{ kHz}.$$

So before downsampling, frequencies above approximately 8 kHz should be removed.

The normalized cutoff relative to the original sample rate is

$$\omega_c = 2\pi \frac{8000}{48000} = \frac{\pi}{3}.$$

Thus

$$\omega_c = \frac{\pi}{M}.$$

This is a good example because it connects the abstract normalized-frequency formula to a real sampling rate.

8. Downsampling versus decimation

These terms are sometimes casually used as synonyms, but the slide deck makes an important distinction.

Downsampling

$$x[n] \rightarrow \downarrow M$$

means simply keeping every M th sample.

Decimation

means

$$\text{lowpass filtering} \rightarrow \downarrow M.$$

The deck explicitly describes the anti-aliasing lowpass filter as the engineering implementation needed before discarding samples.

So a good classroom distinction is:

Downsampling is the mathematical operation. Decimation is the properly engineered rate-reduction system.

9. Why the anti-aliasing filter must come first

Suppose we attempted

$$x[n] \rightarrow \downarrow M \rightarrow H_{AA}(z).$$

This does not work.

Once downsampling has caused frequency components to overlap, the output no longer contains enough information to determine which component came from which original frequency.

Filtering afterward cannot undo that ambiguity.

The order must therefore be

$$x[n] \rightarrow H_{AA}(z) \rightarrow \downarrow M.$$

A very good answer to

Why can't we filter after downsampling?

would be:

Because aliasing is irreversible. After two frequency components fold onto the same output frequency, a later filter cannot distinguish their origins.

10. Ideal versus practical cutoff frequency

For decimation by M , the theoretical limiting frequency is

$$\omega_c = \frac{\pi}{M}.$$

But a real FIR filter cannot transition instantaneously from gain 1 to gain 0.

So in practice, we need something like

$$\omega_p < \frac{\pi}{M}$$

for the passband edge, with a stopband that reaches sufficient attenuation by approximately the aliasing boundary. The slide explicitly points out this transition-band issue.

For example, if

$$M = 2,$$

then the theoretical boundary is

$$\frac{\pi}{2}.$$

A practical design might use

$$\omega_p = 0.42\pi$$

and

$$\omega_s = 0.50\pi.$$

The region

$$0.42\pi < |\omega| < 0.50\pi$$

is the transition band.

11. Why sharper filters cost more

A narrower transition band generally requires a longer FIR filter.

Very approximately,

$$N \propto \frac{1}{\Delta\omega},$$

where $\Delta\omega$ is the transition width.

So if we demand a very abrupt transition, the filter may require many coefficients.

That means:

- more multiplications,
- more storage,
- more latency.

This is why practical multirate design is always a tradeoff.

The slide summarizes that tradeoff as

sharper transition $\Rightarrow$ longer FIR $\Rightarrow$ more delay and computation.

12. Decimation-by-2 example

For

$$M = 2,$$

the output Nyquist limit corresponds to the original normalized frequency

$$\frac{\pi}{2}.$$

So frequencies in

$$\frac{\pi}{2} < |\omega| \leq \pi$$

must be sufficiently suppressed before the samples are thrown away.

Good question

Suppose the input contains components at

$$0.2\pi, \quad 0.35\pi, \quad 0.75\pi.$$

Which components are safe before downsampling by 2?

A good answer:

0.2π and 0.35π are below the new Nyquist boundary $\pi/2$, so they can be retained.
The 0.75π component must be removed by the anti-aliasing filter or it will fold into the output band.

You can make the example stronger by asking:

Where does 0.75π alias?

For $M = 2$,

$$\omega_{\text{out}} = 2(0.75\pi) = 1.5\pi.$$

Modulo 2π ,

$$1.5\pi \equiv -0.5\pi.$$

So it appears at the Nyquist boundary in the output.

13. Why linear-phase FIR filters are attractive

The slides give four main reasons.

Constant group delay

For a linear-phase FIR,

$$H(e^{j\omega}) = A(\omega)e^{-j\omega n_0}.$$

The phase is linear:

$$\phi(\omega) = -\omega n_0.$$

Therefore group delay is

$$\tau_g(\omega) = -\frac{d\phi}{d\omega} = n_0.$$

All retained frequency components experience the same delay.

That means relative phase relationships are preserved.

14. Why phase matters

Suppose a waveform contains

$$x[n] = \cos(0.2\pi n) + 0.5\cos(0.4\pi n).$$

The shape of the waveform depends not only on the amplitudes of those sinusoids but also on their relative phase.

If a filter delays the 0.2π component differently from the 0.4π component, the reconstructed waveform shape changes.

A linear-phase filter avoids that relative phase distortion.

A strong student explanation would be:

Linear phase does not mean zero delay. It means all passband frequencies experience the same delay.

That distinction is important.

15. FIR stability

Finite impulse response filters have no recursive feedback.

Therefore, with finite coefficients, they are BIBO stable.

That makes them attractive in multirate systems.

However, the deck correctly adds an important nuance:

Linear-phase FIR filters are useful, not mandatory.

For example, an IIR filter may achieve comparable magnitude selectivity with far fewer coefficients.

So if latency or computation is dominant, an IIR or minimum-phase design may be preferable.

16. FIR delay

For a length- N symmetric linear-phase FIR, the nominal delay is

$$D = \frac{N - 1}{2}$$

input samples.

For example, if

$$N = 41,$$

then

$$D = 20$$

input samples.

If the original sample rate is 48 kHz,

$$T_D = \frac{20}{48000} \approx 0.417 \text{ ms.}$$

That may be unimportant in offline signal processing but quite important in real-time audio, communications, control, or hearing systems.

17. Concept check: largest safe decimation factor

The slide asks:

A signal occupies $|\omega| \leq 0.22\pi$. What is the largest integer M that can be used without aliasing?

The alias-free requirement is

$$0.22\pi \leq \frac{\pi}{M}.$$

Cancel π :

$$0.22 \leq \frac{1}{M}.$$

Thus

$$M \leq \frac{1}{0.22}.$$

Therefore

$$M \leq 4.545.$$

The largest integer is

$$\boxed{M = 4}.$$

That matches the suggested answer in the deck.

What makes a good answer?

Best answer:

$M = 4$, because the retained signal bandwidth must satisfy $\omega_{\max} \leq \pi/M$. Since $0.22\pi \leq \pi/M$, $M \leq 4.54$, so the largest integer is 4.

Acceptable answer:

Incorrect reasoning:

5, because $1/0.22 \approx 4.5$ and we round to the nearest integer.

The integer must satisfy the inequality. We therefore round **down**, not to the nearest integer.

18. The CNN connection

This is where the lecture becomes particularly useful for students who have seen CNNs without thinking of them as signal-processing systems.

The slide represents a strided convolution as

learned filtering $\rightarrow \downarrow M$.

For a 1-D convolution with stride M ,

$$y[m] = \sum_k h[k]x[mM - k].$$

Define the ordinary convolution

$$v[n] = \sum_k h[k]x[n - k].$$

Then

$$y[m] = v[mM].$$

So mathematically,

stride- M convolution = convolution followed by downsampling by M .

That is not merely an analogy. It is an exact DSP interpretation.

19. Why a learned CNN kernel is not automatically an anti-aliasing filter

In classical decimation, we deliberately design

$$H_{AA}(e^{j\omega})$$

so that energy above the new Nyquist limit is strongly suppressed.

A CNN kernel is optimized for something such as classification loss:

$$\min_{\theta} \mathcal{L}(f_{\theta}(x), y).$$

There is normally no explicit condition saying

$$|H(e^{j\omega})| \approx 0 \quad \text{for} \quad |\omega| > \frac{\pi}{M}.$$

Therefore the learned kernel may preserve or even amplify high-frequency feature content.

If we then apply stride M , that feature content aliases.

This is the major DSP insight behind the ML section of the lecture.

20. Why convolution alone is shift equivariant

Ignoring boundaries, if

$$y[n] = (h * x)[n],$$

and the input is shifted:

$$x_s[n] = x[n - n_0],$$

then

$$(h * x_s)[n] = y[n - n_0].$$

So shifting the input simply shifts the output.

That property is called **shift equivariance**.

Notice that equivariance is not invariance.

Equivariance means:

$$\text{shift input} \Rightarrow \text{shift output}.$$

Invariance would mean:

shift input $\Rightarrow$ same output.

Those are different concepts.

21. Why subsampling breaks exact equivariance

Suppose

$$y[n] = x[2n].$$

Now shift the input by one sample:

$$x_s[n] = x[n - 1].$$

The downsampled result becomes

$$y_s[n] = x_s[2n] = x[2n - 1].$$

The original output used

$$x[0], x[2], x[4], x[6], \dots$$

while the shifted output uses

$$x[-1], x[1], x[3], x[5], \dots$$

These are entirely different sampling phases.

The slide calls attention to exactly this issue: subsampling chooses a sampling phase, and a one-sample input shift may move the feature energy to a different phase.

22. A striking example of shift sensitivity

Consider

$$x[n] = (-1)^n.$$

This is the highest-frequency discrete-time sinusoid:

$$x[n] = \cos(\pi n).$$

Downsample by 2:

$$y[n] = x[2n].$$

Then

$$y[n] = (-1)^{2n} = 1.$$

So the output is

$$1, 1, 1, 1, \dots$$

Now shift the input by one sample:

$$x_s[n] = x[n - 1].$$

Then

$$x_s[n] = -(-1)^n.$$

Downsample:

$$y_s[n] = -1, -1, -1, -1, \dots$$

A one-sample shift of the input changes the output from all +1 to all -1.

That is an excellent classroom example of why high-frequency content and downsampling create severe phase sensitivity.

23. Why aliasing makes CNN shift sensitivity worse

Stride itself reduces resolution, so perfect sample-by-sample shift equivariance cannot generally be maintained.

But aliasing makes the problem worse.

If feature maps contain substantial energy near the Nyquist frequency, tiny shifts can change which phase of those oscillations is sampled.

Consequently the downsampled representation may change dramatically.

The practical consequence described in the deck is that feature maps or predictions may change more than expected after very small translations or time shifts.

For image CNNs, this means moving an object by one pixel can produce unexpectedly different activations.

For audio CNNs, shifting a waveform by one sample can change feature values much more than we would expect perceptually.

24. Anti-aliased CNNs

The classical DSP solution is

lowpass → downsample.

So one natural CNN modification is

learned convolution → nonlinearity → lowpass → downsample.

That is the BlurPool-style design presented in the slides.

The lowpass filter might be something simple such as

$$h[n] = \frac{1}{4} \{1, 2, 1\}.$$

Its frequency response is lowpass-like and attenuates rapidly varying feature content before subsampling.

25. Why BlurPool is not automatically better

There is a subtle tradeoff.

Suppose high-frequency content contains important task information.

Examples include:

- a sharp edge in an image,
- a transient onset in audio,
- a narrow pulse,
- a fine texture,
- a high-frequency modulation pattern.

Aggressive lowpass filtering may suppress these useful features.

So the goal is not

eliminate all high frequencies.

The goal is

control the frequencies that cannot be represented after the planned downsampling.

That is why the deck correctly presents anti-aliased CNNs as a design pattern rather than a universal solution.

26. Mini-demo: three stride-2 systems

The deck proposes comparing:

A. Raw subsampling

```
y = x[:, ::2]
```

This performs no filtering.

It is most vulnerable to aliasing.

B. Learned stride-2 convolution

```
torch.nn.Conv1d(..., stride=2)
```

This performs learned filtering and downsampling together.

Its behavior depends entirely on what kernel training produces.

C. Blur + stride

Conceptually:

```
x_blur = lowpass(x)  
y = x_blur[:, ::2]
```

This deliberately suppresses frequencies that would alias.

The slide asks students to shift the input by one sample and compare the systems.

Good prediction

For a signal with significant energy near the new Nyquist limit:

The lowpass-plus-stride version should usually be the least sensitive to a one-sample shift because the rapidly varying components that produce strong sampling-phase dependence have been attenuated.

That is a better answer than:

BlurPool is always invariant to shifts.

It is not.

The filtering reduces alias-driven sensitivity; it does not magically make the entire network perfectly shift invariant.

27. A good PyTorch demonstration

A simple input could be a combination of low- and high-frequency components:

```
import torch
import numpy as np

n = torch.arange(128)

x = (
    torch.cos(0.15 * np.pi * n)
    + 0.6 * torch.cos(0.80 * np.pi * n)
)

x_shift = torch.roll(x, 1)
```

The high-frequency component at 0.8π is especially useful because it is above the stride-2 Nyquist limit

$$\frac{\pi}{2}.$$

Compare

```
y1 = x[:,2]
y1_shift = x_shift[:,2]
```

against a lowpass filtered version.

For example,

```
h = torch.tensor([0.25, 0.5, 0.25])
```

```
def blur(x):
    x3 = x.view(1, 1, -1)
    h3 = h.view(1, 1, -1)
    return torch.nn.functional.conv1d(x3, h3, padding=1).view(-1)
```

```
y2 = blur(x)[:,2]
y2_shift = blur(x_shift)[:,2]
```

Then compute something like

```
torch.mean(torch.abs(y1 - y1_shift))
torch.mean(torch.abs(y2 - y2_shift))
```

The important teaching point is not that the second quantity must always be smaller for every possible signal.

The point is to show that removing poorly represented high-frequency content can reduce sampling-phase sensitivity.

28. A deeper interpretation for ML students

Traditional DSP says:

Before lowering the sample rate, make the signal compatible with the new Nyquist limit.

CNN architecture design often says:

Reduce spatial or temporal resolution so that later layers require less computation and can represent increasingly abstract features.

Those statements describe the same mathematical event from two different communities.

The DSP perspective raises a useful ML architecture question:

What spectrum exists in the learned feature map immediately before stride?

If the answer includes substantial energy above the post-stride Nyquist limit, aliasing should be expected.

That question is valuable because it gives students a principled way to reason about CNN architecture rather than treating stride as just another hyperparameter.

29. Good answer to the exit question

The deck asks:

Why can a perfectly reasonable learned convolution still be a poor anti-aliasing filter?

The deck's suggested answer is that training does not force the learned kernel to suppress frequencies above the post-stride Nyquist limit.

A strong answer would be:

A CNN kernel is optimized to reduce task loss, not to satisfy a lowpass specification. Therefore it may pass significant feature energy above the Nyquist limit associated with its stride, and that energy will alias during subsampling.

An excellent answer might add:

The network could potentially learn to tolerate some aliasing, but nothing in an ordinary strided convolution guarantees classical anti-alias behavior.

A weak answer would be:

Because CNN filters are not DSP filters.

That misses the important point. CNN kernels **are** filters. The issue is what objective determines their coefficients.

30. Additional concept questions that work well during the lecture

Question 1

A signal is bandlimited to

$$|\omega| \leq 0.3\pi.$$

Can it be directly downsampled by $M = 3$ without aliasing?

We need

$$0.3\pi \leq \frac{\pi}{3}.$$

Since

$$0.3\pi < 0.333\pi,$$

yes.

Good answer

Yes. The signal bandwidth is below the new Nyquist limit $\pi/3$.

Question 2

Can the same signal be downsampled by $M = 4$?

We would require

$$0.3\pi \leq \frac{\pi}{4} = 0.25\pi.$$

This is false.

Good answer

No. Frequencies between 0.25π and 0.3π would alias unless they were first removed.

Question 3

If an anti-aliasing filter removes frequencies above $\pi/4$, can we now downsample that signal by 4?

Yes.

That question helps students understand that the allowable M depends on the bandwidth **after filtering**, not necessarily the bandwidth of the original raw signal.

31. A useful conceptual trap

Ask:

If I downsample a signal by $M = 2$ and then use an ideal lowpass filter to remove the aliases, have I reconstructed the properly decimated signal?

The correct answer is **no**.

Once spectral components alias together,

$$X_1(\omega) + X_2(\omega)$$

appears as one combined output.

A post-filter cannot know which part came from the original in-band signal and which part came from the folded out-of-band signal.

This question strongly reinforces why anti-aliasing must happen **before** downsampling.

32. A useful CNN concept trap

Ask:

If Conv1d uses stride 2 and a kernel of length 9, does the large kernel guarantee anti-aliasing?

No.

Filter length alone says nothing about whether the frequency response is actually lowpass.

A learned kernel could be:

- lowpass,
- highpass,
- bandpass,
- differentiator-like,
- oscillatory,
- or some complicated task-specific response.

A good answer is:

Anti-aliasing depends on the learned frequency response, not merely the kernel size.

33. What students should leave Lecture 11 understanding

The five most important ideas are:

1. Downsampling by M means

$$x_d[n] = x[nM].$$

1. It reduces the sample rate to

$$\frac{F_s}{M}.$$

1. In frequency, spectral copies are stretched, shifted, and added.
2. Therefore signal content must normally be restricted to approximately

$$|\omega| < \frac{\pi}{M}$$

before the rate reduction.

1. A strided CNN convolution is mathematically a filter followed by subsampling, but the learned filter is not automatically an anti-aliasing lowpass filter.

That final point is the most important DSP-to-ML bridge in this lecture.

Week 6, Lecture 12

Explain in more detail what's going on here and what good answers and/or examples are. For all mathematical equations, use standard single dollar signs $\dots$ for inline math and double dollar signs

...

for block/display math. Do not use $(\dots)$ or $[\dots]$. Output the raw text without escaping the dollar signs.

Chat GPT (Medium):

Below is a more detailed instructor-level explanation of Lecture 12, following the structure and terminology of the slide deck. The lecture is built around one central idea: **upsampling creates a denser sampling grid, but zero insertion alone does not reconstruct meaningful intermediate values**. The lowpass interpolation filter is what turns the zero-stuffed sequence into a useful higher-rate signal.

Lecture 12: Upsampling, Interpolation, and Rational Rate Changes

The lecture connects four ideas:

1. Upsampling by zero insertion
2. Interpolation by lowpass filtering
3. Rational sample-rate conversion by L/M
4. Transposed convolution as learned upsampling

The most important conceptual question is:

After inserting zeros, what must the interpolation filter do?

A complete answer is:

It must remove the spectral images created by zero insertion, and its DC gain must be L so that the signal amplitude is preserved.

That two-part answer is the thread that ties together the first half of the lecture.

1. Upsampling by zero insertion

Upsampling by an integer factor L is defined by

$$x_u[n] = \begin{cases} x[n/L], & n = 0, \pm L, \pm 2L, \dots \\ 0, & \text{otherwise.} \end{cases}$$

In words:

Insert $L - 1$ zeros between every pair of original samples.

For example, suppose

$$x[n] = \{2, 4, 1, -1, \dots\}$$

and we upsample by

$$L = 3.$$

Then

$$x_u[n] = \{2, 0, 0, 4, 0, 0, 1, 0, 0, -1, 0, 0, \dots\}.$$

The original samples are still present. Their indices have simply moved:

$$x[k] \rightarrow x_u[kL].$$

The deck correctly emphasizes that the output sample rate becomes LF_s , the original samples remain at $n = kL$, and the zeros are only placeholders.

A very important teaching point is:

Upsampling is not interpolation.

Upsampling creates a higher-rate sequence, but most of its newly created sample positions are zeros.

Interpolation is the subsequent filtering operation that generates useful intermediate values.

2. What happened to physical time?

Suppose the original sampling rate is

$$F_s = 8 \text{ kHz.}$$

Upsample by

$$L = 3.$$

The new sampling rate is

$$F_{s,\text{new}} = 24 \text{ kHz.}$$

Originally,

$$T_s = \frac{1}{8000}.$$

After upsampling,

$$T_{s,\text{new}} = \frac{1}{24000} = \frac{T_s}{3}.$$

So the output grid has three times as many possible sample locations per second.

But only every third location initially contains an original sample.

That gives students a good physical picture of what zero insertion is doing.

3. A good conceptual question

Ask:

If I upsample by $L = 4$, have I created three new measurements between each original pair of samples?

The correct answer is:

No. We have created three new sample positions, but their initial values are zeros. No new signal information has yet been estimated.

A particularly good student answer would add:

The interpolation filter uses neighboring original samples to determine meaningful values at those intermediate positions.

4. Frequency-domain effect of upsampling

For upsampling by L ,

$$X_u(e^{j\omega}) = X(e^{j\omega L}).$$

This equation is deceptively compact.

The key effect is that the original spectrum is **compressed by a factor of L** on the normalized-frequency axis.

The deck emphasizes this as compression followed by spectral images.

Suppose the original signal occupies

$$|\omega| \leq 0.6\pi.$$

If

$$L = 3,$$

then after upsampling the baseband copy occupies approximately

$$|\omega| \leq \frac{0.6\pi}{3} = 0.2\pi.$$

So the spectrum becomes narrower.

5. Why spectral images appear

Students often understand compression but are confused by the extra copies.

The DTFT is periodic:

$$X(e^{j(\omega+2\pi)}) = X(e^{j\omega}).$$

Now substitute $L\omega$:

$$X(e^{jL\omega}).$$

As ω runs through a single interval

$$-\pi \leq \omega \leq \pi,$$

the argument $L\omega$ runs through

$$-L\pi \leq L\omega \leq L\pi.$$

That spans L complete DTFT periods.

Therefore the compressed spectrum appears L times.

For $L = 3$, three copies appear over one 2π interval. That is the concept check in the slide deck.

6. A useful frequency example for $L = 3$

Suppose the original spectrum is confined roughly to

$$|\omega| \leq 0.3\pi.$$

After $L = 3$ upsampling, the desired baseband occupies

$$|\omega| \leq 0.1\pi.$$

But additional images appear around approximately

$$\omega = \pm \frac{2\pi}{3}.$$

The exact shape depends on the original spectrum, but the important picture is:

- one desired copy centered at $\omega = 0$,
- extra compressed copies elsewhere in the $[-\pi, \pi]$ interval.

These extra copies are not desired signal content.

They are artifacts caused by the zero-insertion process.

7. Why do images appear if we only inserted zeros?

This is a very good classroom question.

The best answer is:

Because changing the sampling grid changes the normalized-frequency representation. The inserted zeros alter the sequence structure, and the DTFT becomes $X(e^{jL\omega})$, which traverses the original periodic spectrum L times over one output-frequency interval.

A weaker answer would be:

Because zero insertion makes copies of the spectrum.

That is factually useful, but it does not explain why.

8. Interpolation = upsample + lowpass filter

The complete interpolator is

$$x[n] \rightarrow \uparrow L \rightarrow H_I(z) \rightarrow y[n].$$

The lowpass filter is called an **anti-imaging filter**.

Its job is to preserve the desired baseband spectral copy while suppressing the additional images created by zero insertion.

For ideal interpolation, the cutoff is

$$\omega_c = \frac{\pi}{L}.$$

Why?

Because after upsampling, the useful baseband copy occupies one of L compressed spectral regions, and the first unwanted images lie beyond the boundary associated with π/L .

9. Why $\omega_c = \pi/L$

Suppose the original signal could occupy the entire interval

$$-\pi \leq \omega \leq \pi.$$

After upsampling by L , the desired copy is compressed into

$$-\frac{\pi}{L} \leq \omega \leq \frac{\pi}{L}.$$

So the ideal interpolation filter is

$$H_I(e^{j\omega}) = \begin{cases} L, & |\omega| \leq \pi/L \\ 0, & \text{otherwise.} \end{cases}$$

Notice the gain is not 1.

It is L .

That is one of the most important details in this lecture.

10. Why the interpolation filter needs gain L

The deck uses the best possible teaching example here: a constant signal.

Suppose

$$x[n] = 1$$

for every n .

Upsample by $L = 3$.

Then

$$x_u[n] = \{1,0,0,1,0,0,1,0,0, \dots\}.$$

The average value is now

$$\frac{1}{3}.$$

If we smooth this sequence using a lowpass filter with DC gain 1, the average remains approximately

$$\frac{1}{3}.$$

But that is wrong.

The interpolated version of the constant signal should still be

$$y[n] = 1.$$

Therefore the lowpass filter needs DC gain

$$H_I(e^{j0}) = L.$$

For $L = 3$,

$$H_I(e^{j0}) = 3.$$

That restores the constant signal to its original amplitude.

11. A very useful conceptual check

Ask:

Why doesn't zero insertion itself reduce the amplitudes of the original samples?

Correct answer:

The original nonzero samples remain unchanged. The issue is that they now occur only once every L output samples, so the average or DC level of the high-rate sequence is reduced by L .

That distinction is important.

The gain- L correction is not compensating for the amplitude of each retained sample.

It compensates for their reduced density on the new grid.

12. Time-domain interpretation of interpolation

After zero insertion,

$$x_u[n]$$

contains zeros at the new locations.

Then

$$y[n] = h_l[n] * x_u[n].$$

Each output sample becomes a weighted combination of neighboring original samples.

The deck describes the filter as reconstructing a smoother signal on a denser sampling grid.

This is very closely related to the classical sampling-theorem reconstruction formula.

For ideal bandlimited interpolation,

$$y[n]$$

can be thought of as sinc interpolation of the old samples onto the finer grid.

13. Example: simple linear interpolation intuition

Suppose the original sequence is

$$x[n] = \{0, 2, 4\}.$$

Upsample by $L = 2$:

$$x_u[n] = \{0, 0, 2, 0, 4\}.$$

An ideal or well-designed interpolation filter produces meaningful values between the originals.

A simple linear-interpolation approximation might produce

$$y[n] = \{0, 1, 2, 3, 4\}.$$

That is not exactly what a generic FIR lowpass must produce, but it gives students an intuitive picture:

The zeros are placeholders. The filter replaces them with values consistent with a smooth bandlimited signal.

14. What a good interpolation FIR needs

The deck gives three practical checks.

1. Correct cutoff

The useful compressed baseband must be preserved:

$$|\omega| \leq \omega_p < \frac{\pi}{L}.$$

2. Sufficient image attenuation

The unwanted spectral replicas must be strongly attenuated.

3. Gain L

The coefficients should satisfy approximately

$$\sum_n h[n] = L.$$

Since

$$H(e^{j0}) = \sum_n h[n],$$

this ensures DC gain L .

15. A very practical coding check

If students design an interpolation filter in Python, tell them to feed it

```
x = np.ones(...)
```

and check the steady-state output away from filter transients.

If interpolation is correct,

$$y[n] \approx 1.$$

If the steady-state output is

$$\frac{1}{L},$$

they likely forgot to scale the FIR coefficients by L .

This is one of the simplest debugging tools for interpolation code.

16. Filter delay

For a linear-phase FIR of length N ,

$$D = \frac{N - 1}{2}$$

high-rate samples.

The phrase **high-rate samples** matters.

Suppose

$$N = 31$$

and the output rate after interpolation is

$$48 \text{ kHz.}$$

Then

$$D = 15$$

samples, corresponding to

$$T_D = \frac{15}{48000} = 0.3125 \text{ ms.}$$

Students often incorrectly compute delay using the original sampling rate.

After upsampling, the FIR operates at the higher intermediate rate.

17. Rational sample-rate conversion

Now suppose we want a non-integer rate change.

We use

$$\frac{F_{s,\text{out}}}{F_{s,\text{in}}} = \frac{L}{M}.$$

The basic chain is

$$x[n] \rightarrow \uparrow L \rightarrow H(z) \rightarrow \downarrow M \rightarrow y[n].$$

This is the rational-rate-conversion structure shown in the slides.

The output sampling rate is

$$F_{s,\text{out}} = \frac{L}{M} F_{s,\text{in}}.$$

18. Why upsample first?

This is one of the most valuable conceptual questions in the lecture.

Suppose we want a conversion by

$$\frac{L}{M}.$$

Why not downsample first?

The deck gives the key interpretation:

Upsampling first creates a sufficiently fine intermediate grid from which the desired output samples can be selected.

A good deeper answer is:

The final output sample times generally do not coincide with the original input sample times. Upsampling creates a finer conceptual grid so interpolation can estimate the signal at those required locations before the final decimation chooses the desired subset.

19. A simple rational example

Suppose

$$F_s = 30 \text{ kHz}$$

and we want

$$F_{s,\text{out}} = 20 \text{ kHz}.$$

Then

$$\frac{20}{30} = \frac{2}{3}.$$

So

$$L = 2, \quad M = 3.$$

The conceptual chain is

$$30 \text{ kHz} \rightarrow \uparrow 2 \rightarrow 60 \text{ kHz} \rightarrow \downarrow 3 \rightarrow 20 \text{ kHz}.$$

The intermediate filtering occurs on the 60 kHz grid.

20. Why reduce L/M first?

Suppose someone writes

$$\frac{6}{9}.$$

This represents the same rate change as

$$\frac{2}{3}.$$

Using $L = 6$ and $M = 9$ would create unnecessarily high intermediate rates and extra computation.

So always reduce the ratio to lowest terms:

$$\frac{L}{M} = \frac{L/\text{gcd}(L, M)}{M/\text{gcd}(L, M)}.$$

This is an easy but useful implementation habit.

21. One filter performs two jobs

The filter between $\uparrow L$ and $\downarrow M$ is both:

- an anti-imaging filter,
- an anti-aliasing filter.

The slide deck emphasizes that the stricter of the two constraints determines the cutoff.

The interpolation condition says

$$\omega_c \leq \frac{\pi}{L}.$$

The decimation condition says

$$\omega_c \leq \frac{\pi}{M}.$$

Therefore

$$\omega_c = \min\left(\frac{\pi}{L}, \frac{\pi}{M}\right)$$

for the idealized filter.

In practice, the passband ends somewhat below this value to provide a realizable transition band.

22. Why the minimum?

Suppose

$$L = 3, \quad M = 2.$$

Then

$$\frac{\pi}{L} = \frac{\pi}{3}$$

and

$$\frac{\pi}{M} = \frac{\pi}{2}.$$

The more restrictive limit is

$$\frac{\pi}{3}.$$

So the filter must use approximately

$$\omega_c = \frac{\pi}{3}.$$

Now suppose

$$L = 2, \quad M = 5.$$

Then

$$\frac{\pi}{L} = \frac{\pi}{2}$$

but

$$\frac{\pi}{M} = \frac{\pi}{5}.$$

The decimation requirement is more restrictive, so

$$\omega_c = \frac{\pi}{5}.$$

A good student answer is:

The filter must satisfy both imaging and aliasing constraints, so we choose whichever cutoff is smaller.

23. Important normalization issue

The deck explicitly warns that the cutoff above is expressed relative to the **high-rate intermediate sampling grid**.

This is an easy place for students to make mistakes.

Suppose

$$F_{s,\text{in}} = 30 \text{ kHz},$$

$$L = 2,$$

and

$$M = 3.$$

The intermediate rate is

$$F_{s,\text{int}} = 60 \text{ kHz}.$$

If the normalized cutoff is

$$\omega_c = \frac{\pi}{3},$$

then the corresponding physical frequency is

$$f_c = \frac{\omega_c}{2\pi} F_{s,\text{int}}$$

so

$$f_c = \frac{1}{6} (60 \text{ kHz}) = 10 \text{ kHz}.$$

That agrees with the Nyquist frequency of the final 20 kHz sampling rate.

This is a very good example for showing students that normalized frequency always depends on the sampling rate being used.

24. Real example: 48 kHz to 44.1 kHz

The slide deck uses the important audio conversion

$$48,000 \rightarrow 44,100.$$

The ratio is

$$\frac{44100}{48000} = \frac{147}{160}.$$

Therefore

$$L = 147, \quad M = 160.$$

The conceptual chain is

$$\uparrow 147 \rightarrow \text{LPF} \rightarrow \downarrow 160.$$

The conceptual intermediate sampling rate is

$$48,000 \times 147 = 7,056,000 \text{ Hz.}$$

That sounds extremely inefficient.

And if implemented literally, it would be.

25. Why we do not actually generate all those zeros

A literal implementation would create huge numbers of intermediate samples and then discard most of them.

Polyphase filtering avoids that.

The essential idea is:

Only compute the filtered samples that will actually survive the final downsampling stage.

So the conceptual structure remains

$$\uparrow L \rightarrow H(z) \rightarrow \downarrow M,$$

but the implementation reorganizes the FIR computations to avoid unnecessary work.

This is exactly why routines such as

`scipy.signal.resample_poly`

are so useful.

A good practical explanation is:

The zero-insert/filter/downsample diagram explains the mathematics. Polyphase filtering explains how we implement the same operation efficiently.

26. A good Python example

```
from scipy.signal import resample_poly
```

```
y = resample_poly(x, up=147, down=160)
```

Students should understand what this means mathematically:

$$x[n] \rightarrow \uparrow 147 \rightarrow H(z) \rightarrow \downarrow 160.$$

They should not think `resample_poly` is doing an unrelated numerical trick.

It is an efficient implementation of the same multirate system.

27. Transposed convolution: the ML connection

The final part of the lecture maps interpolation to learned upsampling.

For stride greater than 1, a transposed convolution can be interpreted approximately as

$$\text{zero insertion} \rightarrow \text{learned convolution}.$$

That is the connection emphasized in the deck.

In ordinary DSP interpolation,

$$x[n] \rightarrow \uparrow L \rightarrow h_l[n].$$

In a transposed convolution,

$$x[n] \rightarrow \uparrow L \rightarrow h_\theta[n],$$

where $h_\theta[n]$ contains learned coefficients.

This is another very direct DSP-to-ML connection.

28. Why "transposed convolution" is the better term

Students often hear the term **deconvolution**.

That name is misleading.

A transposed convolution is generally not an inverse of convolution.

If an ordinary convolution is represented by a matrix

$$\mathbf{y} = \mathbf{H}\mathbf{x},$$

then the transposed-convolution operation involves

$$\mathbf{H}^T.$$

That does not imply

$$\mathbf{H}^T = \mathbf{H}^{-1}.$$

So the slide is right to recommend the word **transposed convolution** instead of deconvolution.

29. What can go wrong with transposed convolution?

The deck highlights two related problems:

1. uneven overlap,
2. spectral imaging.

These are related but not identical.

Uneven overlap

Some output positions may receive contributions from more kernel placements than others.

That can create periodic amplitude patterns.

Spectral imaging

Zero insertion creates spectral replicas.

A learned kernel is not automatically a good anti-imaging lowpass.

Therefore some of those images can remain in the output.

Both mechanisms can produce periodic artifacts.

30. A simple uneven-overlap example

Suppose the stride is

$$s = 2$$

but the kernel size is

$$K = 3.$$

As the transposed-convolution kernel moves across the zero-inserted input, some output positions can receive contributions from two neighboring kernel applications while others receive only one.

If the kernel weights are not carefully arranged, this produces an alternating pattern.

In 1-D audio, this can sound like periodic modulation or a tonal buzz.

In 2-D images, the analogous structure can appear as checkerboard texture.

That is why the deck discusses kernel/stride combinations with more even overlap.

31. Why the DSP interpretation helps explain checkerboard artifacts

A pure deep-learning description might say:

Transposed convolutions sometimes create checkerboard artifacts.

The DSP interpretation gives a more satisfying explanation:

We are increasing sample density through a zero-insertion-like operation, so spectral images are expected. If the learned filter does not suppress those images well, periodic structure can remain.

That is exactly analogous to ordinary interpolation.

The main difference is that the interpolation kernel is learned rather than explicitly designed from a lowpass specification.

32. A good conceptual question

Ask:

Is every transposed convolution an interpolation filter?

A strong answer is:

It performs learned upsampling, but it is not necessarily a proper anti-imaging interpolator. Its kernel is optimized for task loss, so its frequency response may not strongly suppress the spectral images created by the stride.

That is much better than saying simply:

Yes, because it increases the sample rate.

33. DSP-informed initialization

The deck suggests initializing the transposed-convolution kernel with a smooth lowpass or linear-interpolation-like response.

The reasoning is:

1. Zero insertion produces images.
2. A lowpass filter suppresses those images.
3. Therefore starting the learned kernel with lowpass-like behavior is a sensible prior.
4. Gradient descent can then modify the coefficients as needed.

This does not guarantee artifact-free output.

It simply starts optimization from a physically reasonable interpolation operator.

34. Example of a simple interpolation-like kernel

For upsampling by 2, a basic triangular kernel might resemble

$$h[n] = \left\{ \frac{1}{2}, 1, \frac{1}{2} \right\}.$$

Its coefficient sum is

$$\frac{1}{2} + 1 + \frac{1}{2} = 2.$$

That is exactly the required DC gain for

$$L = 2.$$

This makes it a nice teaching example because students can immediately connect

$$\sum_n h[n] = L$$

to the interpolation gain requirement.

35. A useful ML comparison

You can ask students to compare these three architectures:

A. Transposed convolution

ConvTranspose1d(stride=2)

B. Nearest-neighbor upsampling + convolution

Upsample → Conv1d

C. Explicit zero insertion + lowpass + convolution

The question is:

Which architecture makes the DSP interpolation assumptions most explicit?

The answer is C.

A good discussion answer might be:

Transposed convolution learns both the interpolation behavior and the feature transformation simultaneously. Explicit lowpass interpolation separates the anti-imaging requirement from the learned feature extraction.

36. Good exit-ticket answer

The deck asks:

Why does an interpolation filter need gain L rather than unity gain?

The deck's answer is that only one out of every L high-rate samples initially contains an original value, so unity-gain smoothing would reduce the constant-signal level by a factor L .

A very good answer is:

Zero insertion reduces the average value of a constant sequence from 1 to $1/L$ because only every L th output sample is nonzero. The interpolation lowpass therefore requires DC gain L so that a constant input remains constant after interpolation.

An excellent answer might add:

Equivalently, the interpolation FIR should satisfy $\sum_n h[n] = L$.

37. Additional concept questions with good answers

Question 1

A sequence is upsampled by

$$L = 4.$$

How many zeros are inserted between adjacent original samples?

Answer:

$$L - 1 = 3.$$

Good explanation:

The original samples occur every fourth sample on the new grid.

Question 2

What is the new sampling rate if

$$F_s = 12 \text{ kHz}$$

and

$$L = 5?$$

Answer:

$$F_{s,\text{new}} = 5(12 \text{ kHz}) = 60 \text{ kHz}.$$

Question 3

What is the ideal interpolation-filter cutoff for

$$L = 4?$$

Answer:

$$\omega_c = \frac{\pi}{4}.$$

Question 4

What should the interpolation filter's DC gain be for

$$L = 4?$$

Answer:

$$H_I(e^{j0}) = 4.$$

38. A very good conceptual trap

Ask:

If the original samples already have the correct amplitude after zero insertion, why do we multiply the interpolation filter by L ?

The best answer is:

Because the issue is not the amplitude of the retained samples; it is their density. A constant signal becomes one nonzero sample followed by $L - 1$ zeros, so its average value drops to $1/L$. The filter gain restores the correct DC level.

This question reveals whether the student truly understands the gain argument.

39. Rational-conversion concept question

Suppose

$$F_{s,\text{in}} = 36 \text{ kHz}$$

and

$$F_{s,\text{out}} = 24 \text{ kHz.}$$

Then

$$\frac{F_{s,\text{out}}}{F_{s,\text{in}}} = \frac{24}{36} = \frac{2}{3}.$$

Therefore

$$L = 2, \quad M = 3.$$

The ideal combined-filter cutoff is

$$\omega_c = \min\left(\frac{\pi}{2}, \frac{\pi}{3}\right) = \frac{\pi}{3}.$$

A strong answer would explicitly say:

The decimation requirement is more restrictive than the interpolation requirement.

40. Another useful rational-rate example

Suppose we want to convert

$$32 \text{ kHz} \rightarrow 48 \text{ kHz.}$$

Then

$$\frac{48}{32} = \frac{3}{2}.$$

So

$$L = 3, \quad M = 2.$$

The conceptual intermediate rate is

$$32 \times 3 = 96 \text{ kHz.}$$

Then

$$\frac{96}{2} = 48 \text{ kHz.}$$

The ideal cutoff is

$$\omega_c = \min\left(\frac{\pi}{3}, \frac{\pi}{2}\right) = \frac{\pi}{3}.$$

Here the interpolation requirement is more restrictive.

41. Good "why" question about order

Ask:

Why not implement L/M conversion as $\downarrow M$ followed by $\uparrow L$?

A good answer:

Because downsampling first may destroy signal information through aliasing before we have created the fine grid needed for the desired output sample positions. Upsampling first allows interpolation at the intermediate rate, after which we can safely select the required output samples.

An even better answer would mention that special reorderings are sometimes possible using multirate identities, but the conceptual sample-rate-conversion chain is $\uparrow L$, filtering, then $\downarrow M$.

42. Good transposed-convolution question

Ask:

If `ConvTranspose1d(stride=2)` increases output length, does that mean it has learned an ideal interpolation filter?

No.

A good answer:

No. It has learned a kernel that is useful for the training objective. Unless the architecture or loss constrains it, there is no guarantee that the kernel behaves as a lowpass anti-imaging filter.

43. Another useful ML example

Consider a generator producing audio at progressively higher sample rates.

Suppose one layer approximately doubles the temporal resolution.

A transposed-convolution layer effectively performs something like

$$\uparrow 2 \rightarrow h_\theta[n].$$

If $h_\theta[n]$ fails to attenuate the image near π , energy can remain at periodic high-frequency locations.

After several layers, those artifacts can become structured tonal components.

This gives students a DSP explanation for why some generated audio has a faint buzzing or periodic texture.

44. What students should understand by the end

The core mathematical facts are:

Upsampling:

$$x_u[n] = \begin{cases} x[n/L], & n = kL \\ 0, & \text{otherwise} \end{cases}$$

and

$$X_u(e^{j\omega}) = X(e^{jL\omega}).$$

Interpolation:

$$\uparrow L \rightarrow H_I(z)$$

with approximately

$$\omega_c = \frac{\pi}{L}$$

and

$$H_I(e^{j0}) = L.$$

Rational conversion:

$$\uparrow L \rightarrow H(z) \rightarrow \downarrow M$$

with

$$F_{s,\text{out}} = \frac{L}{M} F_{s,\text{in}}$$

and idealized cutoff

$$\omega_c = \min\left(\frac{\pi}{L}, \frac{\pi}{M}\right).$$

The ML bridge is:

$\text{ConvTranspose1d} \approx \text{zero insertion} + \text{learned filtering}$.

The important conceptual insight is that **learned upsampling does not eliminate classical sampling theory**. Zero insertion still creates images, and the learned kernel still has to deal with them somehow. The DSP framework explains both why interpolation filters are designed as they are and why learned upsampling layers can develop periodic artifacts.

Week 6, Lectures 11 and 12, Exit Tickets

What are good exit ticket questions with answers for these lectures?

Gemini Pro:

Week6_Lecture11_Downsampling_Decimation_CNN_Strides_2.pptx

- **Question:** In one sentence: why can a perfectly reasonable learned convolution still be a poor anti-aliasing filter?
- **Answer:** A learned convolution kernel is optimized for task loss rather than being explicitly constrained to function as a lowpass anti-aliasing filter. Without this constraint, the kernel may fail to suppress high-frequency content, allowing aliased features to pass through and causing the output to vary sharply under small shifts.

Week6_Lecture12_Upsampling_Interpolation_Rational_Rate_Changes_2.pptx

- **Question:** Why does an interpolation filter need gain L rather than unity gain?
 - **Answer:** Upsampling by zero insertion introduces $L - 1$ zeros between original samples, which reduces the average value of the sequence to $1/L$. A lowpass filter with a unit DC gain would preserve this reduced average rather than restoring the signal. Applying a gain of L (i.e., setting $H_f(e^{j0}) = L$) ensures that the reconstructed sequence preserves the original amplitude, keeping constant inputs constant.
-

Week 6, Lectures 11 and 12, Exit Tickets

What are good exit ticket questions with answers for these lectures?

ChatGPT: (Medium)

Here are exit-ticket questions that fit these two Week 6 lectures well. I would keep the actual student exit ticket to about 2–3 questions per lecture, then rotate among the alternatives below across sections or semesters.

For Lecture 11, the strongest questions should test whether students understand that downsampling is not just “dropping samples,” but a sampling operation with a frequency-domain consequence, and that a strided CNN is mathematically related to filtering followed by subsampling.

Lecture 11 — Downsampling, Decimation, and CNN Strides

Question 1 — Core concept

A signal is downsampled by a factor of M . What must be true about its bandwidth before downsampling if aliasing is to be avoided?

Good answer:

The signal should be limited approximately to

$$|\omega| < \frac{\pi}{M}$$

before downsampling. In practice, an anti-aliasing lowpass filter is used with a transition band below the ideal boundary.

What you are looking for:

- Recognition of the new Nyquist limit.
- Recognition that filtering happens **before** downsampling.
- Better answers mention a practical transition band.

This directly targets the alias-free condition emphasized in the lecture.

Question 2 — Small calculation

A signal occupies

$$|\omega| \leq 0.18\pi.$$

What is the largest integer downsampling factor M that can be used without aliasing?

Solution:

Require

$$0.18\pi \leq \frac{\pi}{M}.$$

Therefore

$$M \leq \frac{1}{0.18} \approx 5.56.$$

So

$$M = 5.$$

Good answer:

$M = 5$, because the signal bandwidth must remain below π/M .

Common weak answer:

$M = 6$ because 5.56 rounds to 6.

This is a useful diagnostic because students must respect the inequality rather than round normally.

Question 3 — Downsampling versus decimation

What is the difference between downsampling and decimation?

Good answer:

Downsampling simply keeps every M th sample. Decimation first applies an anti-aliasing lowpass filter and then downsamples.

A very strong answer would write

$$\text{decimation} = \text{lowpass filtering} + \downarrow M.$$

This distinction is central to the middle portion of Lecture 11.

Question 4 — Why filtering first?

Why can we not simply downsample first and remove the aliased frequencies afterward with a lowpass filter?

Good answer:

Once frequency components alias onto the same output frequency, they have been added together and cannot be uniquely separated. Therefore anti-alias filtering must happen before downsampling.

Excellent answer:

Aliasing is an irreversible many-to-one mapping of frequency components, so post-filtering cannot determine which portion of the observed output came from the original baseband and which portion folded into it.

This is one of the best conceptual exit-ticket questions because it reveals whether students understand what aliasing actually means.

Question 5 — Frequency-domain interpretation

Describe what happens to the spectrum when a sequence is downsampled by M .

Good answer:

The spectrum is stretched by M , shifted copies appear, and those copies are added. If they overlap, aliasing occurs.

A concise phrase from the lecture is:

Stretch, shift, add.

That wording matches the mental model emphasized in the deck.

Question 6 — CNN connection

Why is a Conv1d layer with stride= M a multirate DSP operation?

Good answer:

A stride- M convolution can be interpreted as ordinary convolution followed by keeping every M th output sample, so it is filtering followed by downsampling.

Mathematically, if

$$v[n] = (h * x)[n],$$

then the strided output is

$$y[m] = v[mM].$$

This is the main DSP-to-ML bridge in Lecture 11.

Question 7 — CNN anti-aliasing

Why can a learned convolutional kernel with stride 2 be a poor anti-aliasing filter even if the CNN achieves high classification accuracy?

Good answer:

The kernel is trained to minimize task loss, not to satisfy a lowpass frequency-response requirement. It may therefore pass high-frequency feature energy above the post-stride Nyquist limit, causing aliasing.

This is essentially the exit question already present in the deck and is an excellent choice.

Question 8 — Shift equivariance

Why can aliasing in a strided CNN make the network sensitive to a one-sample input shift?

Good answer:

Downsampling selects one sampling phase. A one-sample shift changes which phase is sampled, and high-frequency or aliased feature content can therefore produce a substantially different output.

Excellent answer:

Convolution alone is shift equivariant, but subsampling reduces spatial or temporal resolution. Aliased high-frequency features make the result especially sensitive to the chosen sampling phase.

The lecture explicitly distinguishes inherent resolution loss from additional alias-driven sensitivity.

My preferred Lecture 11 exit ticket

If I wanted only three questions, I would use:

1. A signal occupies $|\omega| \leq 0.2\pi$. What is the largest safe integer M ?
2. Why must the anti-aliasing filter come before $\downarrow M$?
3. Why is `Conv1d(..., stride=M)` not automatically a proper DSP decimator?

Together these test **calculation, DSP reasoning, and the DSP-to-ML connection**.

For Lecture 12, the most important conceptual targets are different: students should know that zero insertion creates images, that interpolation requires both lowpass filtering and gain L , and that rational conversion combines interpolation and decimation in one chain. The lecture then carries the same logic into transposed convolutions.

Lecture 12 — Upsampling, Interpolation, and Rational Rate Changes

Question 1 — Upsampling versus interpolation

What is the difference between upsampling and interpolation?

Good answer:

Upsampling inserts $L - 1$ zeros between original samples. Interpolation then lowpass-filters that zero-inserted sequence to remove spectral images and generate meaningful intermediate values.

A very good answer explicitly says:

Zero insertion alone is not interpolation.

That distinction is emphasized near the beginning of the lecture.

Question 2 — Frequency-domain effect

What happens to the DTFT when a signal is upsampled by L ?

Good answer:

The DTFT becomes

$$X_u(e^{j\omega}) = X(e^{jL\omega}).$$

The original spectrum is compressed by L , and L spectral copies appear across one 2π interval.

This directly tests the central frequency-domain result.

Question 3 — Number of images

If a sequence is upsampled by $L = 4$, how many compressed spectral copies appear over one 2π interval?

Answer:

4

Good explanation:

Because $X(e^{j4\omega})$ traverses four periods of the original DTFT while ω traverses one 2π interval.

This generalizes the $L = 3$ concept check already in the deck.

Question 4 — Interpolation filter cutoff

What is the ideal cutoff frequency of the interpolation lowpass filter for upsampling by L ?

Answer:

$$\omega_c = \frac{\pi}{L}$$

Good explanation:

Zero insertion compresses the desired baseband copy into approximately $|\omega| \leq \pi/L$. The lowpass filter passes that copy and suppresses the unwanted images.

This is one of the core interpolation design requirements.

Question 5 — Why gain L ?

Why does the interpolation filter require DC gain L rather than unity gain?

Good answer:

After zero insertion, only one out of every L high-rate samples contains an original value. For a constant input, the average therefore becomes $1/L$. A filter with gain L restores the original constant amplitude.

A very strong answer might add

$$H_I(e^{j0}) = L$$

or equivalently

$$\sum_n h[n] = L.$$

This is probably the single best exit-ticket question for Lecture 12 because students commonly remember the lowpass filter but forget the gain.

Question 6 — Rational conversion

A signal must be converted from 30 kHz to 20 kHz. Find L and M and give the conceptual multirate chain.

Solution:

$$\frac{20}{30} = \frac{2}{3}.$$

Therefore

$$L = 2, \quad M = 3.$$

The system is

$$\uparrow 2 \rightarrow H(z) \rightarrow \downarrow 3.$$

Good answer:

Upsample by 2, lowpass filter, then downsample by 3.

Question 7 — Combined filter cutoff

For a rational converter with

$$L = 3, \quad M = 5,$$

what is the idealized cutoff of the combined anti-imaging/anti-aliasing filter?

Solution:

$$\omega_c = \min\left(\frac{\pi}{L}, \frac{\pi}{M}\right).$$

Thus

$$\omega_c = \min\left(\frac{\pi}{3}, \frac{\pi}{5}\right) = \boxed{\frac{\pi}{5}}.$$

Good explanation:

The filter must satisfy both anti-imaging and anti-aliasing requirements, so the more restrictive cutoff is used.

That is the exact design principle emphasized in the rational conversion section.

Question 8 — Why upsample first?

Why is a rational converter normally represented as

$$\uparrow L \rightarrow H(z) \rightarrow \downarrow M$$

rather than downsampling first?

Good answer:

Upsampling first creates the finer intermediate sampling grid needed to represent the desired output sample times. The intermediate filter can then both remove interpolation images and prevent aliasing before the final downsampling.

A strong answer may also say:

Downsampling first could irreversibly alias information before interpolation has taken place.

Question 9 — 48 kHz to 44.1 kHz

What integer ratio is used to convert 48 kHz audio to 44.1 kHz?

Solution:

$$\frac{44100}{48000} = \frac{147}{160}.$$

Therefore

$$L = 147, \quad M = 160.$$

The conceptual converter is

$$\uparrow 147 \rightarrow H(z) \rightarrow \downarrow 160.$$

A good follow-up question is:

Do practical implementations actually generate all the zero-valued samples at the enormous intermediate rate?

Good answer:

No. Polyphase implementations reorganize the computation and calculate only the required outputs.

This matches the practical example in the slides.

Question 10 — Transposed convolution

What is the DSP interpretation of a ConvTranspose1d layer with stride greater than 1?

Good answer:

It can be interpreted approximately as zero insertion followed by convolution with a learned filter, with details depending on padding and indexing.

Symbolically,

$$\uparrow L \rightarrow h_{\theta}[n].$$

The important qualification is that the learned kernel is **not automatically a proper anti-imaging filter**.

Question 11 — Checkerboard or periodic artifacts

Give two reasons transposed convolution can create periodic artifacts.

Good answer:

1. Uneven overlap can cause some output positions to receive more kernel contributions than others.
2. Zero insertion produces spectral images that the learned kernel may not adequately suppress.

For image models these may appear as checkerboard patterns; for audio they may appear as periodic or tonal artifacts.

Question 12 — DSP-informed ML design

Why might initializing a transposed-convolution kernel with a smooth lowpass interpolation filter be useful?

Good answer:

The model begins with an upsampling operation that already suppresses some imaging energy. This provides a physically sensible starting point while still allowing gradient descent to adapt the filter.

A very good answer should **not** claim that this guarantees an artifact-free network. The slides explicitly frame it as an engineering prior rather than a guarantee.

My preferred Lecture 12 exit ticket

For only three questions, I would use:

1. Why does zero insertion create spectral images?
2. Why must an interpolation filter have DC gain L ?
3. For $L = 2$ and $M = 3$, what is the combined ideal cutoff, and why?

The third answer is

$$\omega_c = \min\left(\frac{\pi}{2}, \frac{\pi}{3}\right) = \frac{\pi}{3}.$$

This combination tests **conceptual understanding, amplitude scaling, and rational-rate design**.

Best combined Week 6 exit ticket

If you want one short exit ticket at the end of the entire week, I would use these four:

1. Before downsampling by M , why do we normally lowpass-filter to approximately π/M ?

Good answer:

To remove frequencies that the lower sampling rate cannot represent and prevent them from folding into the preserved band.

2. After upsampling by L , why is lowpass filtering necessary?

Good answer:

Zero insertion compresses the desired spectrum and creates L spectral copies. The lowpass filter keeps the baseband copy and removes the images.

3. Why does the interpolation filter need gain L ?

Good answer:

Zero insertion reduces the DC level of a constant sequence by L , so gain L restores its amplitude.

4. Complete this analogy:

Conv1d with stride $> 1 \leftrightarrow ?$

ConvTranspose1d with stride $> 1 \leftrightarrow ?$

Good answer:

Conv1d stride $\leftrightarrow$ learned filtering + downsampling

and

ConvTranspose1d $\leftrightarrow$ zero insertion + learned filtering.

That final pair is especially useful for this course because it forces students to connect the Oppenheim & Schaffer multirate material to the CNN architecture material rather than treating the DSP and ML portions as separate subjects.
